Excellent 🚀 Let's move on to **Module 16: Subqueries**.

📘 SQL CLASS NOTE – MODULE 16: SUBQUERIES

1. Introduction

A **subquery** (or inner query) is a query **inside another SQL query**.
It allows you to use the result of one query as input for another.

- Can be placed in:
  - `SELECT`
  - `FROM`
  - `WHERE`
  - `HAVING`

👉 Subqueries make SQL **more powerful** by breaking complex problems into smaller queries.

2. Syntax

```
SELECT column1
FROM table1
WHERE column2 IN (SELECT column2 FROM table2 WHERE condition);
```

3. Types of Subqueries

🔹 1. Single-row Subquery

- Returns **one value**.
- Used with operators: `=`, `>`, `<`, `>=`, `<=`.

```sql
-- Find students older than the average age
SELECT FirstName, LastName, Age
FROM Students
WHERE Age > (SELECT AVG(Age) FROM Students);
```

2. Multi-row Subquery

- Returns **multiple values**.
- Used with `IN`, `ANY`, `ALL`.

```sql
-- Find students enrolled in any course taught by 'Dr. Smith'
SELECT FirstName, LastName
FROM Students
WHERE StudentID IN (
    SELECT StudentID
    FROM Enrollments E
    JOIN Courses C ON E.CourseID = C.CourseID
    WHERE C.Teacher = 'Dr. Smith'
);
```

3. Nested Subquery in FROM (Derived Table)

- Subquery acts like a temporary table.

```sql
-- Show average age per grade using a subquery
SELECT GradeLevel, AvgAge
FROM (
    SELECT GradeLevel, AVG(Age) AS AvgAge
    FROM Students
    GROUP BY GradeLevel
) AS GradeStats;
```

4. Correlated Subquery

- Inner query depends on outer query.

```sql
-- Find students who are the youngest in their grade
SELECT StudentID, FirstName, Age, GradeLevel
FROM Students S
WHERE Age = (
    SELECT MIN(Age)
    FROM Students
    WHERE GradeLevel = S.GradeLevel
);
```

4. Why Use Subqueries?

- To **filter dynamically** using results of another query.
- To avoid creating temporary tables.
- To solve **complex queries step by step**.

5. Real-Life Use Cases

- **Schools**: Find students older than class average.
- **E-commerce**: Customers who bought the most expensive product.
- **Banks**: Accounts with balance higher than the bank average.
- **Hospitals**: Patients treated by the same doctor.

6. Classwork (Practice) ✅

👉 Try these queries:

1. Find students older than the **average age** of all students.
2. Find students enrolled in courses taught by **Dr. Smith**.
3. Find the **youngest student(s)** in each GradeLevel using a correlated subquery.
4. Find students who are not enrolled in any course (use a subquery with `NOT IN`).
5. Use a subquery in `FROM` to show the **average age per grade**.
6. Find courses that have more enrollments than the **average enrollments per course**.
7. Find students whose age is **greater than all students in Grade 9**.

7. Challenge Question 💡

👉 Write a query to list students who are enrolled in **more than one course** using a subquery.

(Hint: Use a subquery with `GROUP BY` and `HAVING COUNT(*) > 1`.)

✅ That's the **expanded Module 16: Subqueries**.

Perfect 👍 Let's solve the **Module 16 Classwork + Challenge** step by step with SQL queries.

✅ Solutions to Module 16: SUBQUERIES

1. Find students older than the **average age** of all students.

```sql
SELECT StudentID, FirstName, LastName, Age
FROM Students
WHERE Age > (SELECT AVG(Age) FROM Students);
```

2. Find students enrolled in courses taught by **Dr. Smith**.

```sql
SELECT S.StudentID, S.FirstName, S.LastName
FROM Students S
WHERE S.StudentID IN (
    SELECT E.StudentID
    FROM Enrollments E
    JOIN Courses C ON E.CourseID = C.CourseID
    WHERE C.Teacher = 'Dr. Smith'
);
```

3. Find the **youngest student(s)** in each GradeLevel using a correlated subquery.

```sql
SELECT S.StudentID, S.FirstName, S.LastName, S.Age, S.GradeLevel
FROM Students S
WHERE S.Age = (
    SELECT MIN(Age)
    FROM Students
    WHERE GradeLevel = S.GradeLevel
);
```

4. Find students who are not enrolled in any course.

```sql
SELECT StudentID, FirstName, LastName
FROM Students
WHERE StudentID NOT IN (SELECT StudentID FROM Enrollments);
```

5. Use a subquery in `FROM` to show the **average age per grade.**

```sql
SELECT GradeLevel, AvgAge
FROM (
    SELECT GradeLevel, AVG(Age) AS AvgAge
    FROM Students
    GROUP BY GradeLevel
) AS GradeStats;
```

6. Find courses that have more enrollments than the **average enrollments per course**.

```sql
SELECT C.CourseID, C.CourseName, COUNT(E.StudentID) AS EnrollmentCount
FROM Courses C
JOIN Enrollments E ON C.CourseID = E.CourseID
GROUP BY C.CourseID, C.CourseName
HAVING COUNT(E.StudentID) > (
    SELECT AVG(EnrollCount)
    FROM (
        SELECT COUNT(StudentID) AS EnrollCount
        FROM Enrollments
        GROUP BY CourseID
    ) AS CourseStats
);
```

7. Find students whose age is **greater than all students in Grade 9**.

```sql
SELECT StudentID, FirstName, LastName, Age
FROM Students
WHERE Age > ALL (SELECT Age FROM Students WHERE GradeLevel = 'Grade 9');
```

💡 Challenge Question

👉 List students who are enrolled in **more than one course**.

```sql
SELECT StudentID, FirstName, LastName
FROM Students
WHERE StudentID IN (
    SELECT StudentID
    FROM Enrollments
    GROUP BY StudentID
    HAVING COUNT(CourseID) > 1
);
```

📌 This uses a subquery with `GROUP BY` + `HAVING` to filter students enrolled in multiple courses.